

Reader - writer locks

Problem: Different data structure access might require different kinds of locking.

eg:- concurrent list operations

- inserting new element to the list
- look up
 - ↳ changes the state of the list
 - ↳ simply read the list.

(makes no change in the state of the list)

⇒ as long as we can guarantee that no insert is on-going, we can allow many lookups to proceed concurrently.

How to achieve this? → reader-writer lock.

uses a new pair of synchronization to update the data structure

{ rlock_acquire_writelock() → to acquire a write lock
rlock_release_writelock() → to release it

→ together ensures that only a single writer can acquire the lock and update the critical section at a time.

What about read locks? `rwlock_acquire_readlock()`

internally it happens that → the first reader acquires the lock
the first reader also acquires the writelock → more readers will be
allowed to acquire the readlock too → but writers have to wait
until all readers are finished.

```
1 typedef struct _rwlock_t {  
2     sem_t lock;           // binary semaphore (basic lock)  
3     sem_t writelock;      // allow ONE writer/MANY readers  
4     int  readers;         // #readers in critical section  
5 } rwlock_t;
```

→ to protect 'readers' counter

```
6  
7 void rwlock_init(rwlock_t *rw) {  
8     rw->readers = 0;  
9     sem_init(&rw->lock, 0, 1);  
10    sem_init(&rw->writelock, 0, 1);  
11 }  
12
```

reader acquire function

→ start with no readers.

both semaphores initialized to 1 (unlocked)

* lock = 1 → available for protecting the readers counter

* writelock = 1 → resource is free (no writer/readers)

```
13 void rwlock_acquire_readlock(rwlock_t *rw) {  
14     sem_wait(&rw->lock);  
15     rw->readers++;  
16     if (rw->readers == 1) // first reader gets writelock  
17         sem_wait(&rw->writelock);  
18     sem_post(&rw->lock);  
19 }
```

→ one more reader entering

→ first reader blocks all the subsequent writers.

reader release function

→ release lock for 'readers' (as done updating the counter)

```
20  
21 void rwlock_release_readlock(rwlock_t *rw) {  
22     sem_wait(&rw->lock);  
23     rw->readers--;  
24     if (rw->readers == 0) // last reader lets it go  
25         sem_post(&rw->writelock);  
26     sem_post(&rw->lock);  
27 }
```

→ lock to safely decrement 'readers'

→ allowing waiting writers to proceed

writer acquire/release function

```
28  
29 void rwlock_acquire_writelock(rwlock_t *rw) {  
30     sem_wait(&rw->writelock);  
31 }  
32
```

→ wait for exclusive access

```
33 void rwlock_release_writelock(rwlock_t *rw) {  
34     sem_post(&rw->writelock);  
35 }
```

→ release exclusive access

Behavior summary situation

Readers allowed?

Writers allowed?

No one inside

✓

✓

one or more readers
inside

✓ (more readers
can join)

X

a writer inside

X

X (no one else
can enter)

How to use it?

when you want to

read shared data

use functions

`pthread_rwlock_acquire_readlock()` → access →
`pthread_rwlock_release_readlock()`

write shared data

`pthread_rwlock_acquire_writelock()` → modify →
`pthread_rwlock_release_writelock()`

a short example

```
rwlock_acquire_readlock(&lock);  
int value = shared_data;    → safely read  
rwlock_release_readlock(&lock);  
  
rwlock_acquire_writelock(&lock);  
shared_data = 42;          → safely write  
rwlock_release_writelock(&lock);
```

Question: Is reader-writer lock as defined earlier fair?

No! While a writer is waiting, new readers can enter freely. Consequently, if readers keep arriving rapidly, writers may starve forever.

How to make it fair?

- * once a writer starts waiting, no new readers should enter
- * existing readers finish normally
- * after they finish, the waiting writer should proceed before new readers come in.

Exercise: Think how to implement the above idea!

The Dining Philosophers' problem

↳ one of the most famous concurrency problems posed, and solved by **Dijkstra**

Problem set-up

- There are 5 philosophers sitting around a circular table.
- Between each pair of philosophers is a single fork (and thus 5 forks in total)
- each philosopher have times where they think and where they eat.

↳ no forks needed

↳ needs two forks

(one from left and one from right)



Question: How can we design a system (with synchronization) so that:

- * no two philosophers use the same fork at once (mutual exclusion).
- * the system does not deadlock (everyone waiting forever).
- * no philosopher starves (everyone eventually gets to eat).
- * concurrency is high (as many philosophers can eat at the same time).

Assume that each philosopher has a unique identifier p from 0 to 4.

What should be done?

```
while (1) {  
    think();  
    get_forks(p);  
    eat();  
    put_forks(p);  
}
```

challenge: define these functions!

Attempt 1: To eat $\rightarrow$ we need to acquire two exclusive forks

$\rightarrow$ grab a lock on each of the fork
(one on left and one on right)

when we are done eating $\rightarrow$ release the locks (so that others can use them)

```
1 void get_forks(int p) {  
2     sem_wait(&forks[left(p)]);  
3     sem_wait(&forks[right(p)]);  
4 }  
5  
6 void put_forks(int p) {  
7     sem_post(&forks[left(p)]);  
8     sem_post(&forks[right(p)]);  
9 }
```

$\rightarrow$ refers to the fork on the left side of the philosopher p

```
int left(int p) { return p; }  
int right(int p) { return (p + 1) % 5; }
```

$\rightarrow$ refers to the fork on the right side of the philosopher p .

What is the problem with the above approach?

Deadlock! (think, why?)

Hint: each philosopher grab their left fork before anyone could grab their right!

So, what to do now?

↳ break the dependency.

change how forks are acquired by atleast one of the philosophers.

say, the highest numbered philosopher gets the forks in a different order than the others.

```
1 void get_forks(int p) {  
2     if (p == 4) {  
3         sem_wait(&forks[right(p)]);  
4         sem_wait(&forks[left(p)]);  
5     } else {  
6         sem_wait(&forks[left(p)]);  
7         sem_wait(&forks[right(p)]);  
8     }  
9 }
```

↳ so that cycle is broken!
and atleast one philosopher
can always proceed.

This is still not perfect, as there
is a possible starvation
(some unlucky professor might
keep waiting)

Exercise: Explore more fairer ways for solving the above problem.

Exercise: Also, explore other interesting concurrency problems!

Thread throttling

↳ when OS needs to limit the # threads that can actually run concurrently (even though your program have created many)

Why it happens? eg:- threads share CPU cores.

suppose you have 8 CPU cores and 100 active threads.
Then as only 8 thread can run simultaneously (assuming 1 per core), the OS scheduler keeps switching b/w them → giving each a small time slice.

so, if too many threads are active,

- each thread gets less CPU time
- context switching overhead increases
- performance can actually get worse instead of better.

This leads the OS scheduler throttling the threads to maintain fairness and stability.

Another example: many threads accessing a memory intensive region

↳ sum of memory allocation request exceeds physical memory → system thrashes!

what is the solution? limit the # threads running concurrently.

↳ use 'semaphore' as a 'gatekeeper'

we can use a **counting semaphore** to control how many threads can run at once.

- initialize a semaphore with the maximum allowed concurrency.
- before doing the main work, each thread calls `sem_wait()`.
- after finishing, the thread calls `sem_post()` to release its "slot".

How to implement semaphores? → build our own version of semaphores called "Zemaphores" using locks and condition variables.

```
1  typedef struct __Zem_t {
2      int value;
3      pthread_cond_t cond;
4      pthread_mutex_t lock;
5  } Zem_t;
6
7  // only one thread can call this
8  void Zem_init(Zem_t *s, int value) {
9      s->value = value;
10     Cond_init(&s->cond);
11     Mutex_init(&s->lock);
12 }
13
14 void Zem_wait(Zem_t *s) {
15     Mutex_lock(&s->lock);
16     while (s->value <= 0)
17         Cond_wait(&s->cond, &s->lock);
18     s->value--;
19     Mutex_unlock(&s->lock);
20 }
21
22 void Zem_post(Zem_t *s) {
23     Mutex_lock(&s->lock);
24     s->value++;
25     Cond_signal(&s->cond);
26     Mutex_unlock(&s->lock);
27 }
```

- locks (mutexes) → ensure mutual exclusion
- condition variables → allow waiting and signaling b/w threads
- semaphores → unify mutual exclusion and ordering into one construct.

Zem_t code demonstrates how a semaphore can be implemented using:

- a `pthread_mutex_t` and
- a `pthread_cond_t`